NESTJS DEVELOPER BACKEND PRACTICAL TEST

1. THE CHALLENGE

Build a minimal, functional REST API for a College Course Enrollment System. The application must manage Courses and tracking Student enrollments.

You have full creative freedom regarding the database models, properties, and specific entity relationships—design it in a way that best demonstrates your database architecture skills.

2. TECHNICAL SPECIFICATIONS

- **Framework:** NestJS (TypeScript)
- **Database:** Use either **MySQL** (via TypeORM) OR **MongoDB** (via Mongoose) based on your proficiency.
- **(Good to Have) API Documentation & Testing:** Implement and configure `Swagger UI (@nestjs/swagger)`. All endpoints must be accessible and testable via the `/api` or `/docs` route.
-

3. CORE REQUIREMENTS & LOGIC

Your API must expose endpoints to handle the following functional flows:

1. **Admin Management:** Login / Manage Admin Users
2. **Course Management:** Create new courses and fetch available courses.
3. **Student Management:** Register new student profiles.
4. **Enrollment Engine:** A dedicated process to enroll a student into a course. You must implement logical safeguards to ensure:
    - A student cannot enroll in the exact same course more than once.
    - Enrollment fails gracefully if a course has already reached its maximum capacity.
    -

4. QUALITY & BEST PRACTICES

- Utilize NestJS Built-in `ValidationPipe` with `class-validator` for incoming payloads (DTOs).
- Ensure proper separation of concerns across Controllers, Services, and Modules.
- Handle edge cases and business logic failures gracefully using appropriate HTTP exception filters (e.g., 400 Bad Request, 404 Not Found).

> 💡 **Developer Note:** Focus on robust business logic, schema design, and seamless Swagger integration. Do not waste time building frontend blocks, or complex UI styling.

5. EVALUATION SPLIT

- **40% Core Functionality & Authentication:** Working endpoints easily testable and it should have proper working authentication on Admin readable APIs.
- **30% Business Logic & Data Integrity:** Bulletproof validation against duplicate enrollments and over-capacity handling.
- **20% Code Architecture & Cleanliness:** Effective modular setup, use of DTOs, and asynchronous best practices (async/await).
- **10% Swagger:** If implemented it should allow user to open swagger call APIs.

Submit your clean source code as a GitHub repository link or a compressed ZIP archive (excluding node_modules).